Advanced Topics

Game Design

Why

Bullet hell games are clearly defined by their precise gameplay and increasing difficulty. These two properties are clearly linked to game design. These might ruin the experience if not properly designed and implemented.

What

We used market research to expand on what these properties mean in the context of game design.

- **Precise Gameplay**: We found that most staple games in the genre have smooth controlled movement and accurate hit detection. The controlled movement in combination with fair hit detection allows for satisfying and rewarding near misses.
- **Difficulty Scaling:** Rapid difficulty scaling rewards high skilled players while not deterring newer players . We found that there exist 2 types of difficulty scaling, numerical and variety. The trade off is that while the former is less development heavy, the latter is considered more fun.

How

To create a smooth movement we set our frame rate at 60 fps and allow anti-aliasing for sub-pixel positioning. We also tie movement not to the frame but to the difference in time since last call.By using the *Shape.intersect(Shape)* method we can accurately check whether the player or an enemy is hit. We can optimize this by first checking if the object can be hit and then doing a less expensive bounding check.

Due to our limited scope we decided to scale our difficulty numerically. We scale the enemy spawn interval, shoot interval and speed based on the current score of the player. The score itself is determined by the time passed and enemies eliminated.

What We Learned

Implementing collision detection proved challenging. *Shape.intersect(Shape)* is performance-heavy, therefore bounding box checks were necessary to minimize overhead. Managing collision permission logic also required us to reconsider our code structure.

Finally, our limited resources forced us to use numerical scaling over variety scaling, compromising some quality. Overall, this project deepened our understanding of game design principles and their practical implementation.

Git/ Version Control

Why

We chose Git and GitHub because working together on different laptops quickly gets messy. Since we were developing player movement, enemies, and shooting at the same time, sharing files manually would have led to overwritten progress. Using Git allowed us to work in parallel on our own branches without breaking the working game on main.

What

Through our research into professional Git branching workflows, we focused on three core principles:

- **Feature Branch Isolation:** Using distinct branches (Player, enemy, projectiles, highscore) branched off main guarantees that unstable work-in-progress features do not break the baseline game loop.
- **State & Workspace Management:** Understanding how the working directory, staging index, and remote references interact, particularly using commands like git stash to shelve uncommitted work before pulling remote updates.
- **Class Separation:** Recognizing that decoupling monolithic classes directly reduces merge conflicts by minimizing simultaneous edits to identical source files.

How

We hosted our repository on GitHub and used separate feature branches for each mechanic (such as Player and enemy). To share progress between our computers, we used git fetch and git pull. Whenever local uncommitted changes blocked an incoming pull, we used git stash or resets to keep our workspace clean. Splitting our code into classes like EnemyManager and TextElimination also helped our Git workflow, as we were no longer modifying GamePanel.java at the same time.

What We Learned

The biggest lesson we learned was that good code structure makes version control much easier. In the beginning, having all the rendering and logic inside GamePanel.java caused constant merge conflicts. Refactoring the game into smaller classes (Actor, EnemyManager, ProjectileManager) almost completely removed this problem. Overall, we learned how to solve practical Git issues, sync remote branches properly, and collaborate smoothly without losing work.



